

1. Objectives:

The objectives of this lab project are:

1. To capture and preprocess audio signals to remove noise and enhance speech quality using **bandpass filtering** and amplitude thresholding,
2. To apply **FFT**-based spectral analysis to analyze frequency components in voice signals,
3. To implement **smoothing** via **convolution** to reduce fluctuations and improve speech segment detection,
4. To extract **MFCC** features for compact, perceptually-relevant voice representation and **voice matching** using cosine similarity,
5. To compare input audio with stored templates for accurate voice-based command recognition and authentication,
6. To capture and preprocess images using resizing, grayscale conversion, and normalization for efficient processing,
7. To extract image features via histogram analysis and apply **filtering and smoothing** techniques to improve matching accuracy,
8. To perform image deblurring using Richardson-Lucy **deconvolution** and Gaussian filtering to enhance image clarity,
9. To integrate DSP techniques in both audio and image streams to enable real-time recognition and processing, and
10. To combine DSP-enhanced audio and image analysis with AI logic to build a robust, multimodal voice and image-controlled assistant.

2. Introduction:

This project presents an **advanced voice assistant** that leverages Digital Signal Processing (DSP) techniques to achieve accurate real-time recognition and interaction. The system captures audio from a microphone, preprocesses it using **bandpass filtering**, **FFT**-based spectral analysis, and convolution-based smoothing, and extracts **MFCC** features to represent voice signals compactly. These features are compared with stored templates using cosine similarity, enabling reliable voice command recognition and speaker authentication.

In addition to audio processing, the system captures and preprocesses images using resizing, grayscale conversion, and normalization, then extracts key features via histogram analysis. Image deblurring is performed using Richardson-Lucy **deconvolution** combined with **Gaussian filtering** and sharpening, allowing clear and accurate image matching against stored templates.

The project integrates real-time DSP operations for both audio and image signals, combined with **AI-driven logic for decision-making**. Users **can record and match voice commands, capture and match images, deblur stored images, perform mathematical and physics calculations, search knowledge bases, and open apps or websites through voice commands**. Overall, the system demonstrates the power of DSP in enhancing signal quality, extracting meaningful features, and enabling a robust, multimodal interactive assistant.

The overall code implemented for this project was 703 lines written in **python** . So we shall attach the whole code at end. Implementing this huge project surely required some AI assistance on the way but the credit must be given to our original idea, our deep understanding of dsp & almost 40 work hours to successfully build this advanced model.

The python libraries used for this project are:

- **speech_recognition** → audio capture & speech-to-text.
- **pyttsx3** → TTS (text-to-speech).
- **numpy** → array math, DSP operations.
- **scipy.signal** → filters, convolution.
- **scipy.spatial.distance.cosine** → cosine similarity for voice/image matching.
- **sympy** → math solver, derivatives, integrals.
- **re, os, shutil, subprocess, webbrowser** → file & system operations.
- **requests, BeautifulSoup, googlesearch, wikipedia** → web searching.
- **cv2** → image capture & processing.
- **librosa** → audio processing, MFCC extraction.
- **skimage.restoration.richardson_lucy** → image deconvolution.
- **scipy.ndimage.gaussian_filter** → image smoothing.

let's now explain the process part by part along with the results

3. Initialization

```
recognizer = sr.Recognizer()
app_cache = {}
app_index = {}
MEMORY_FILE = "memory.json"
IMAGE_MEMORY_FILE = "image_memory.json"
AUDIO_MEMORY_FILE = "audio_memory.json"
RECORDED_AUDIO_DIR = "recorded_audio"
CAPTURED_IMAGE_DIR = "captured_images"
```

Fig 1: Initialization code

This sets up speech recognizer, memory caches, and directories for audio/image storage & ensures persistent knowledge (voice, image, text).

4. TTS Setup

```
def speak(text):
    try:
        engine = pyttsx3.init()
        engine.setProperty('rate', 150)
        engine.setProperty('volume', 1.0)
        engine.say(text)
        engine.runAndWait()
    except Exception as e:
        print(f"⚠ TTS Error: {e}")
```

Fig 2: TTS (text to speech) setup

- Converts text → spoken audio.
- rate = words/min (speech speed)
- volume = loudness (0–1).
- Uses internal TTS engine (**pyttsx3**) to output voice.

5. Preprocessing audio:

5.1: Bandpass filter design:

```
def butter_bandpass(lowcut, highcut, fs, order=5):  
    nyq = 0.5 * fs  
    low = lowcut / nyq  
    high = highcut / nyq  
    b, a = butter(order, [low, high], btype='band')  
    return b, a  
  
def bandpass_filter(data, lowcut=300, highcut=3400, fs=16000, order=5):  
    b, a = butter_bandpass(lowcut, highcut, fs, order=order)  
    return lfilter(b, a, data)
```

Fig 3: Butterworth bandpass filter design code

A **Butterworth filter** is a type of signal processing filter designed to have a maximally flat frequency response in the passband. Maximally flat means there are no ripples in the frequency range that the filter allows to pass. It is widely used in audio processing, communications, and DSP because it provides smooth filtering without distortion in the passband.

Here we designed a fifth order Butterworth bandpass filter that cuts frequencies outside human voice range (~300–3400 Hz). For achieving a sharp transition higher filter order is required but it increases the complexity and the width of the transition band. The magnitude response of an n-th order Butterworth filter:

$$|H(f)| = \frac{1}{\sqrt{1 + \left(\frac{f}{f_c}\right)^{2n}}} \quad (5.1.1)$$

Where:

f = frequency of the input signal

f_c = cutoff frequency

n = filter order (controls the sharpness of the cutoff)

The **butter_bandpass** function calculates the coefficients of the filter & **lfilter** implements the filter using those coefficients. **lfilter** applies filter to 1D signal. Each sample of y is computed using the recursive difference equation:

$$y[n] = \frac{1}{a} (b_0 x[n] + b_1 x[n-1] + \dots + b_M x[n-M] - a_1 y[n-1] - \dots - a_N y[n-N]) \text{ where, } M, N = \text{len}(b, a) - 1 \quad (5.1.2)$$

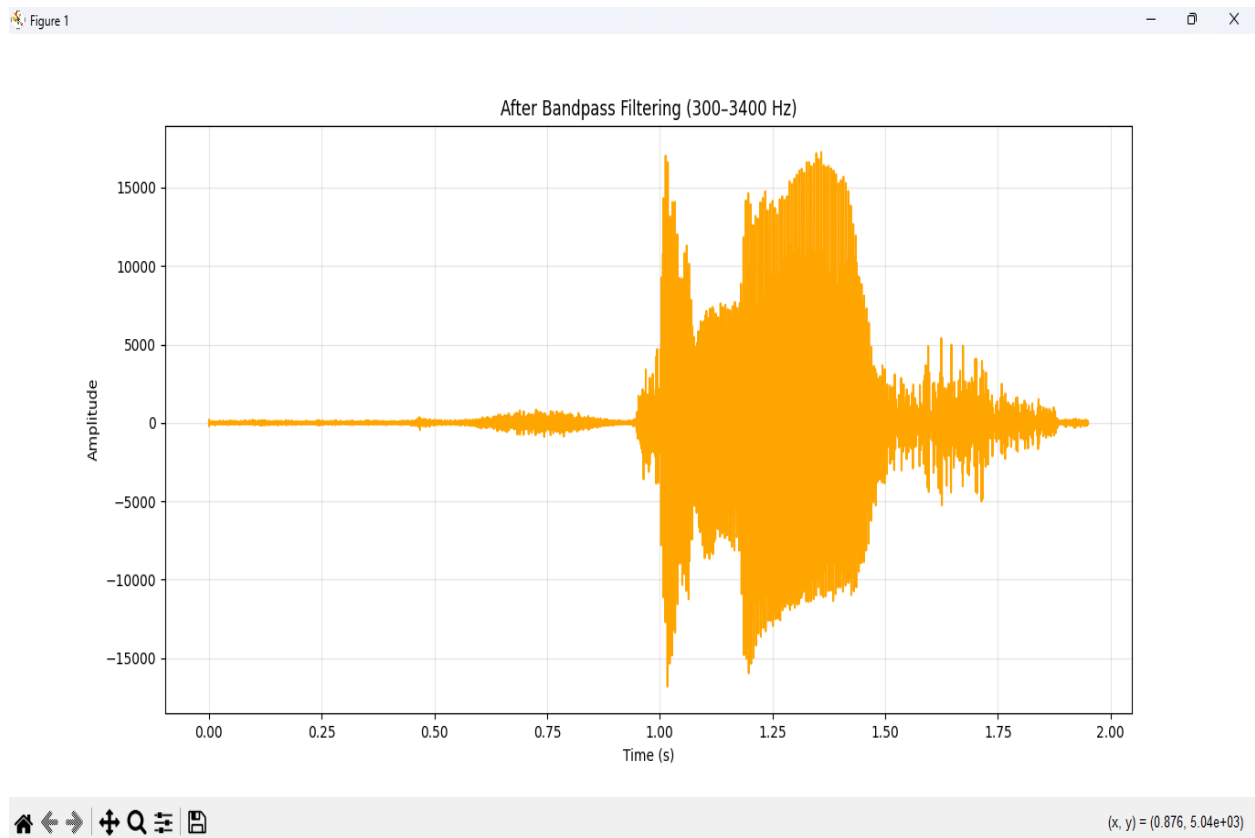
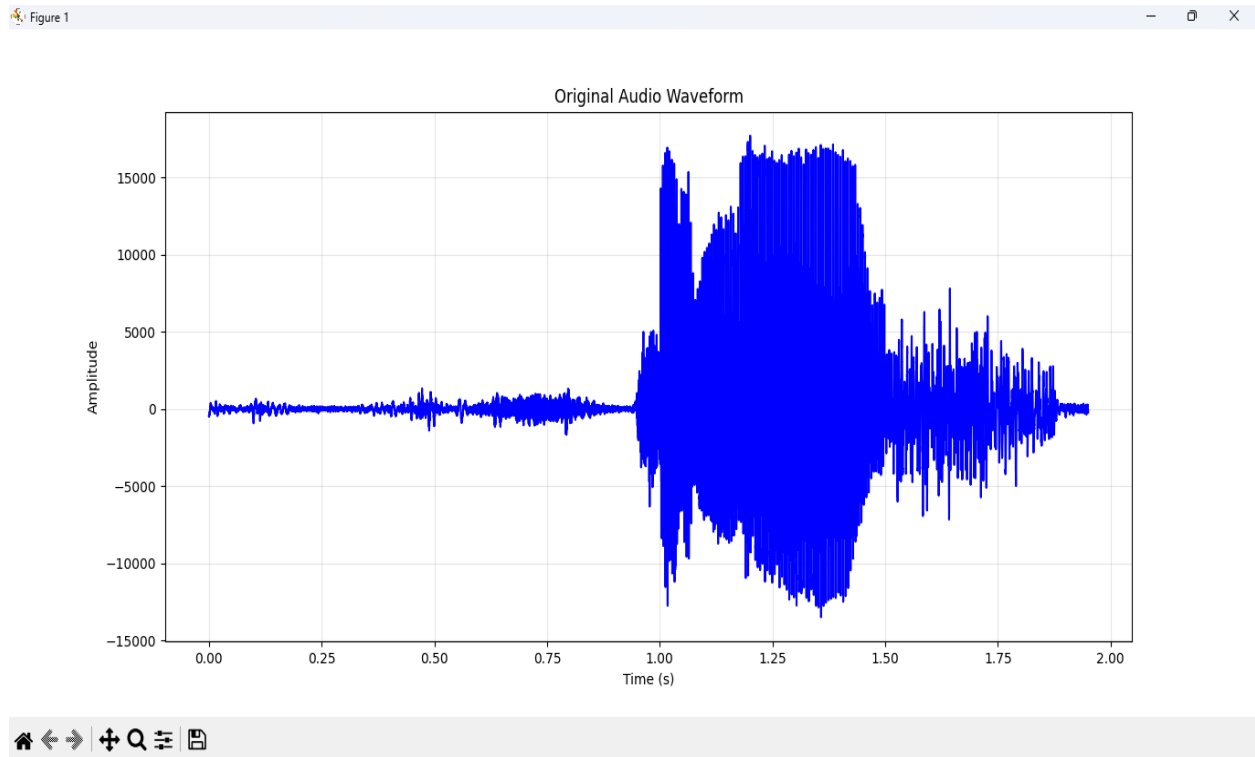


Fig 4: Effect of bandpass filtering on voice signal

5.2 FFT operation:

```
def preprocess_audio(audio_data, sample_rate):  
    try:  
        audio_np = np.frombuffer(audio_data, dtype=np.int16)  
        filtered = bandpass_filter(audio_np, 300, 3400, fs=sample_rate, order=6)  
  
        fft_signal = np.fft.fft(filtered)  
        magnitude = np.abs(fft_signal)  
        threshold = np.max(magnitude) * 0.01  
        fft_signal[magnitude < threshold] = 0  
        filtered_fft = np.fft.ifft(fft_signal).real
```

Fig 5: FFT operation code

FFT (Fast Fourier Transform) → converts audio to frequency domain.

Thresholding → removes weak (low power) frequencies (noise suppression).

Inverse FFT → reconstructs cleaned signal back to time domain.

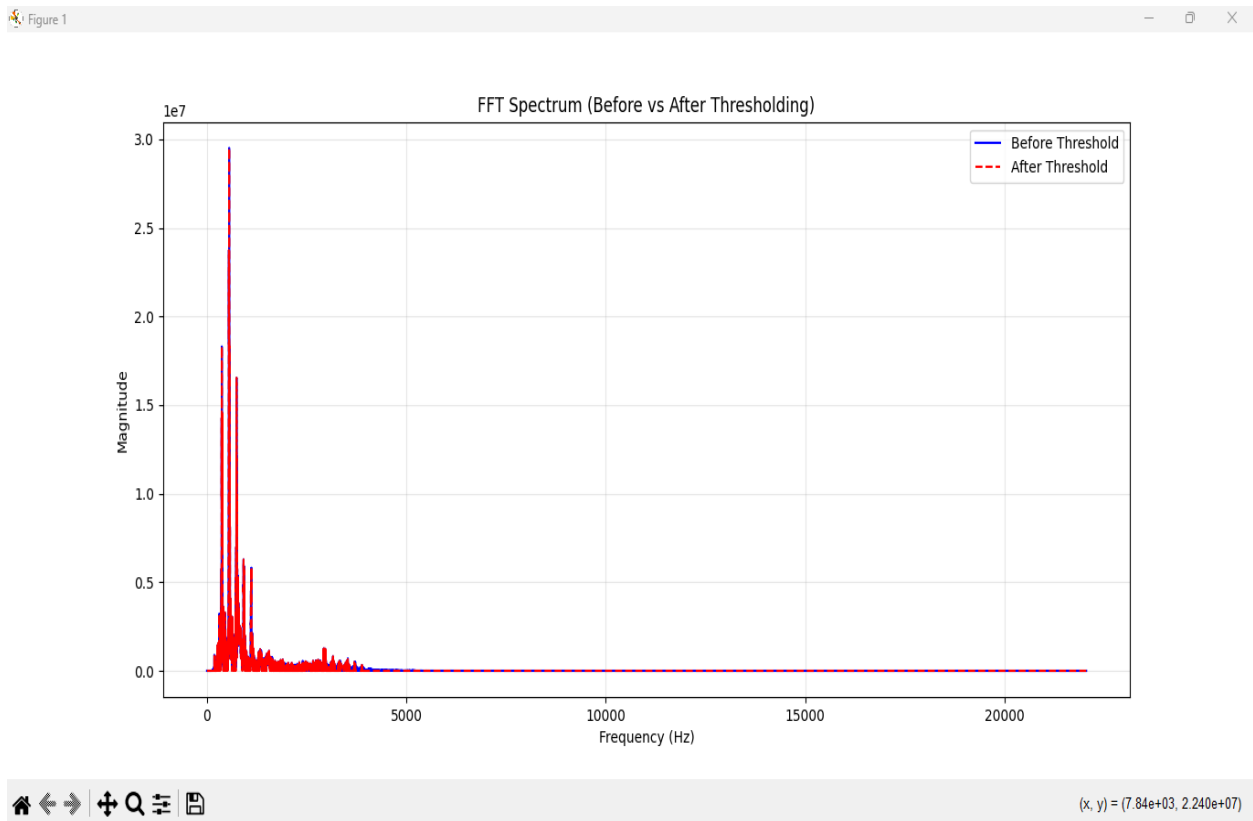


Fig 6: FFT spectrum (before & after thresholding)

5.3 :Moving average filter (convolution):

```
kernel = np.ones(5)/5
smoothed = convolve(filtered_fft, kernel, mode="same")
```

Fig 7: Moving average filtering using convolution

A **kernel** is just a small array of numbers (weights) that is used to transform a signal (1D) or an image (2D). When a kernel is applied using convolution, each output sample is a weighted average of nearby input samples.

A smoothing kernel is a kernel designed to reduce noise or rapid fluctuations. It works like a moving average filter. There are different types of smoothing kernel

- 1.Simple Moving Average (box kernel)
- 2.Weighted Average (triangle kernel) : Gives more weight to the center, less to the edges.
3. Gaussian Kernel: Smooths strongly but preserves shape better than a box filter.

Here Gaussian kernel was used which is most common in dsp usage

5.4: Voice activity detection

```
voice_indices = np.where(np.abs(smoothed) > 500)[0]
if len(voice_indices) > 0:
    smoothed = smoothed[voice_indices[0]:voice_indices[-1]+1]
```

Fig 8: Voice activity detection code

This simply removes silence sections by thresholding amplitude.

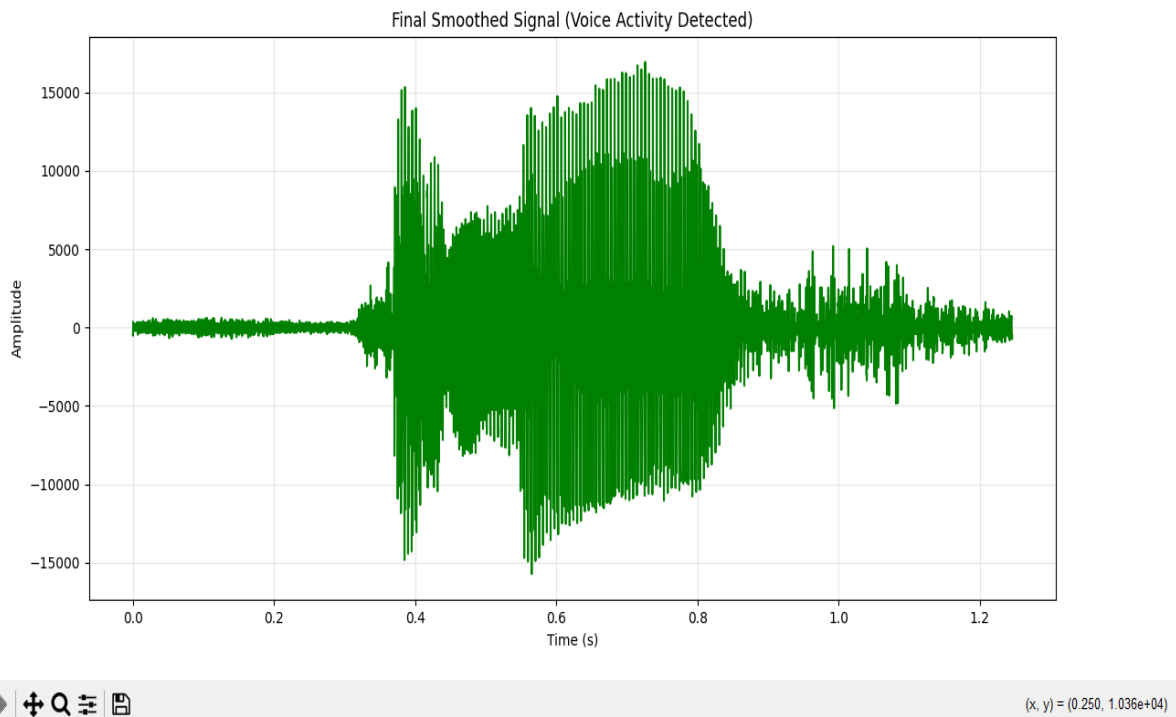


Fig 9: Final smoothed signal

6. Image Processing

6.1: Image capturing: Opens camera → snaps picture → saves and returns it.

```
def capture_image(image_source="webcam"):
    """Capture image from webcam or file, save to file, and display"""
    try:
        if image_source == "webcam":
            cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
            if not cap.isOpened():
                return None, "Failed to open webcam. Ensure it is connected and not in use."
            time.sleep(1)
            ret, frame = cap.read()
            cap.release()
            if not ret:
                return None, "Failed to capture image from webcam"
            # Save to file
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = os.path.join(CAPTURED_IMAGE_DIR, f"captured_{timestamp}.jpg")
            cv2.imwrite(filename, frame)
            # Display on screen
            cv2.imshow("Captured Image", frame)
            cv2.waitKey(0) # Wait for any key press to close
            cv2.destroyAllWindows()
            _, buffer = cv2.imencode('.jpg', frame)
            return buffer.tobytes(), f"Image captured from webcam, saved as {filename}, and displayed on screen"
        else:
            if os.path.exists(image_source):
                img = cv2.imread(image_source)
                if img is None:
                    return None, f"Failed to load image from {image_source}"
                _, buffer = cv2.imencode('.jpg', img)
                return buffer.tobytes(), f"Image loaded from {image_source}"
            return None, f"File {image_source} not found"
    except Exception as e:
        return None, f"Image Capture Error: {e}"
```

Fig 10: Image capturing code

6.2: Feature Extraction:

```
def extract_image_features(image_data):  
    """Extract histogram features for image matching with resized image"""  
    try:  
        img = cv2.imdecode(np.frombuffer(image_data, np.uint8), cv2.IMREAD_GRAYSCALE)  
        # Resize image to reduce processing time (max width/height 300 pixels)  
        max_size = 300  
        height, width = img.shape  
        scale = max_size / max(height, width)  
        if scale < 1:  
            img = cv2.resize(img, (int(width * scale), int(height * scale)), interpolation=cv2.INTER_AREA)  
        hist = cv2.calcHist([img], [0], None, [256], [0, 256])  
        cv2.normalize(hist, hist)  
        return hist.flatten().tolist()  
    except Exception as e:  
        print(f"⚠ Image Feature Extraction Error: {e}")  
        return None
```

Fig 11: Feature extraction code from image

- Removes color info to simplify processing. Converts the image to grayscale
- Pixels are expressed by their intensity values (0–255) called histogram
- Image size is reduced to max 300 pixels (for faster computation).
- Interpolation = weighted averaging → 2D signal resampling
- Counts how many pixels have each intensity (0–255).
- Result: 256-bin vector → feature vector for matching.
- Scales values to the same range (important for comparing images).
- Prevents bright/dark images from biasing the similarity.
- Converts 2D histogram into 1D feature vector

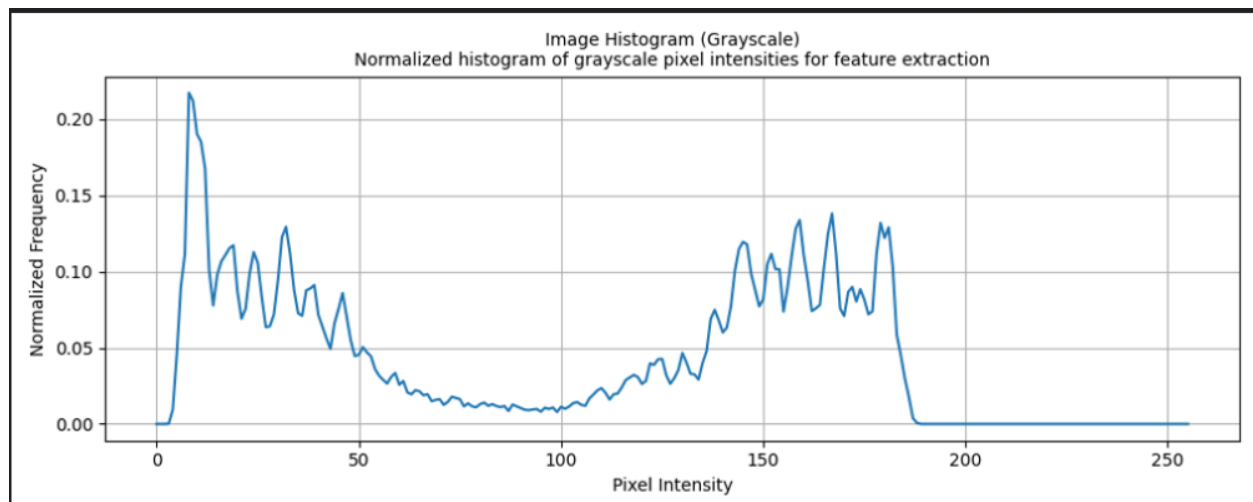


Fig 12: Image histogram

6.3: Image Matching

Image matching is done by **histogram correlation** between the two images. A correlation threshold value is set to 0.8. If the correlation value is > 0.8 then images will match. If lower it won't match.

```
def match_image(image_data):
    """Match input image against stored image features"""
    try:
        input_features = extract_image_features(image_data)
        if input_features is None:
            return None
        input_hist = np.array(input_features, dtype=np.float32)

        image_memory = load_image_memory()
        if not image_memory:
            print("⚠ No image features available for matching")
            return None

        best_match = None
        best_similarity = -np.inf

        for desc, data in image_memory.items():
            stored_hist = np.array(data.get("features"), dtype=np.float32)
            similarity = cv2.compareHist(input_hist, stored_hist, cv2.HISTCMP_CORREL)
            if similarity > best_similarity:
                best_similarity = similarity
                best_match = desc

        return best_match if best_similarity > 0.8 else None
    except Exception as e:
        print(f"⚠ Image Matching Error: {e}")
        return None
```

Fig 13: Code for image matching

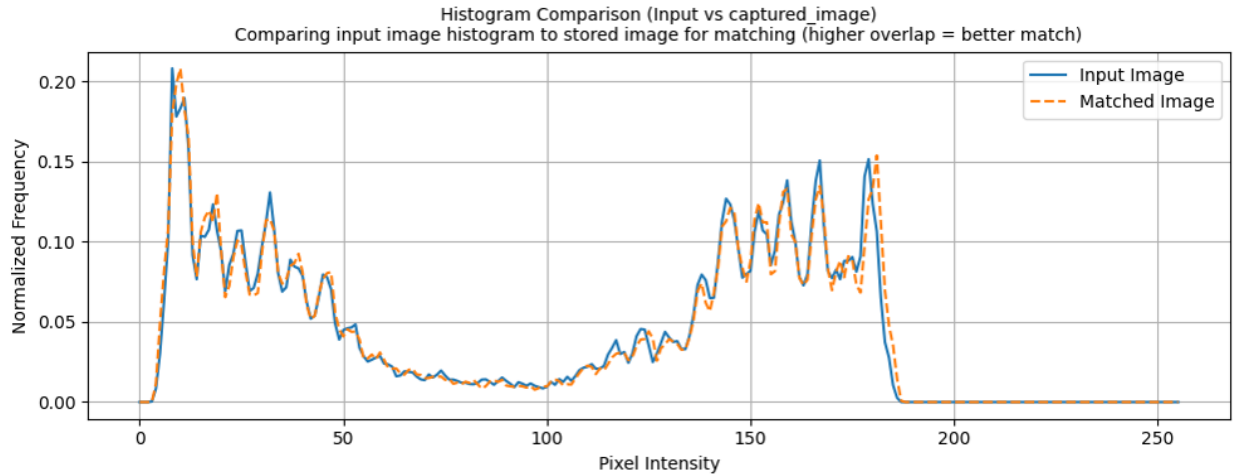


Fig 14: Histogram comparison of two images

6.4: Deblurring Images

In imaging, a blurred image is modeled as:

$$\text{Blurred} = \text{Original} * \text{PSF} + \text{Noise} \quad \text{Here, 2 PSF is a Gaussian kernel}$$

PSF (Point Spread Function) describes how a single point spreads into neighboring pixels due to blur. Using **Richardson-Lucy deconvolution** original image can be recovered from blurred.

$$I_{n+1} = I_n * \left(\frac{I_{\text{blurred}}}{I_n * \text{psf}} * \text{psf}_{\text{flipped}} \right) \quad (6.4.1)$$

How it works:

1. Starts with initial guess (usually blurred image itself)
2. Convolves guess with PSF → compare with observed blurred image
3. Multiplies guess by correction factor → update estimate
4. Repeat for 15 times
5. Each iteration involves 2D convolution with PSF.

After deconvolution, a small convolution kernel is applied to enhance edges. This is **high-pass filtering** (amplifies rapid intensity changes → sharper image)

```
def deblur_image(description):
    """Apply Richardson-Lucy deconvolution to deblur an image from memory and display it"""
    try:
        image_memory = load_image_memory()
        if description not in image_memory:
            return None, f"No image found for description: {description}"
        image_data = bytes.fromhex(image_memory[description]["data"])
        img = cv2.imdecode(np.frombuffer(image_data, np.uint8), cv2.IMREAD_COLOR) # Load in color
        if img is None:
            return None, f"Failed to decode image for description: {description}"

        # Convert to float32 and normalize to [0, 1] for each channel
        img = img.astype(np.float32) / 255.0

        # Create Gaussian blur kernel
        kernel_size = 5
        sigma = kernel_size / 4.0 # Smaller sigma for sharper PSF
        psf = np.zeros((kernel_size, kernel_size))
        center = kernel_size // 2
        for i in range(kernel_size):
            for j in range(kernel_size):
                psf[i, j] = np.exp(-((i - center) ** 2 + (j - center) ** 2) / (2 * sigma ** 2))
        psf /= psf.sum() # Normalize PSF

        # Apply Richardson-Lucy deconvolution to each color channel
        deblurred = np.zeros_like(img)
        for channel in range(img.shape[2]):
            deblurred[:, :, channel] = richardson_lucy(
                img[:, :, channel], psf, num_iter=15, clip=True
            )

        # Clip and scale back to uint8
        deblurred = np.clip(deblurred * 255, 0, 255).astype(np.uint8)

        # Apply sharpening filter
        sharpening_kernel = np.array([
            [0, -1, 0],
            [-1, 5, -1],
            [0, -1, 0]
        ])
        deblurred = cv2.filter2D(deblurred, -1, sharpening_kernel)
    
```

Fig 15: Code to deblur image

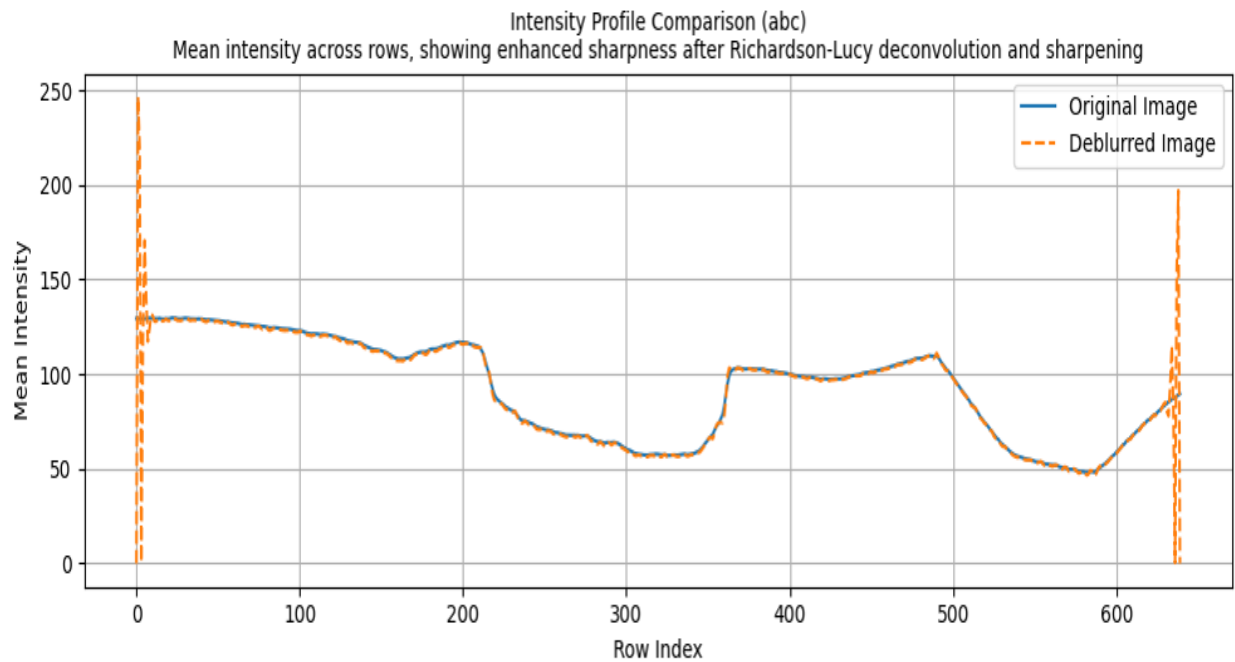


Fig 16: Richardson Lucy deconvolution

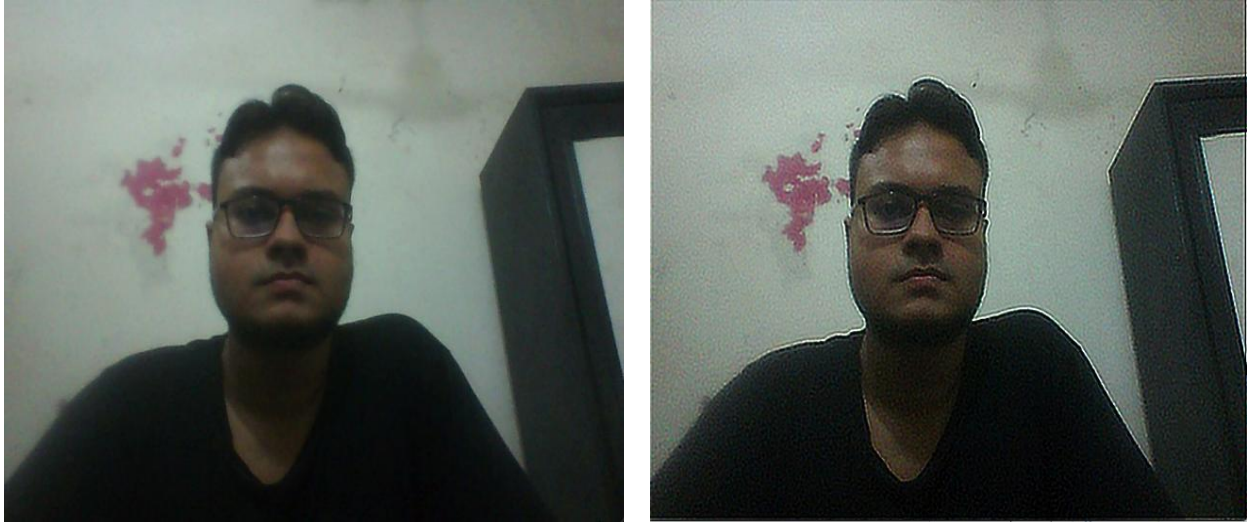


Fig 17: Original image(left) vs deblurred image (right)

6.5: Image Memory (Save & Load)

- Saves images + their extracted features in image memory. json.
- Enables later comparison.
- Creates a small database of images to match against.

```
def save_image_memory(image_data, description):
    """Save processed image and its features to memory"""
    try:
        image_memory = load_image_memory()
        features = extract_image_features(image_data)
        if features is None:
            return f"Failed to save image: Could not extract features"
        image_memory[description] = {
            "data": image_data.hex(),
            "features": features
        }
        with open(IMAGE_MEMORY_FILE, "w") as f:
            json.dump(image_memory, f, indent=2)
        return f"Saved image with description: {description}"
    except Exception as e:
        return f"Failed to save image to memory: {e}"

def load_image_memory():
    """Load image memory from file"""
    try:
        if os.path.exists(IMAGE_MEMORY_FILE):
            with open(IMAGE_MEMORY_FILE, "r") as f:
                image_memory = json.load(f)
            # Validate entries
            valid_memory = {}
            for desc, data in image_memory.items():
                if isinstance(data, dict) and "data" in data and "features" in data:
                    valid_memory[desc] = data
                else:
                    print(f"⚠ Invalid image memory entry for {desc}, discarding")
            return valid_memory
        return {}
    except Exception as e:
        print(f"⚠ Image Memory Load Error: {e}")
        return {}
```

Fig 18: Image saving & loading code

7. Voice processing

7.1: Voice recording

```
# ----- VOICE RECORDING -----
def record_audio(sample_rate=16000):
    """Record audio until silence is detected"""
    try:
        with sr.Microphone(sample_rate=sample_rate) as mic_source:
            recognizer.adjust_for_ambient_noise(mic_source)
            print("Recording... Speak now (stops after silence).")
            audio = recognizer.listen(mic_source, timeout=5, phrase_time_limit=None)
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = os.path.join(RECORDED_AUDIO_DIR, f"recording_{timestamp}.wav")
            with open(filename, "wb") as f:
                f.write(audio.get_wav_data())
            return audio.get_raw_data(), sample_rate, f"Audio recorded and saved as {filename}"
    except Exception as e:
        print(f"⚠ Audio Recording Error: {e}")
        return None, None, f"Failed to record audio: {e}"
```

Fig 19: Code for voice capturing

- Uses microphone for live capture.
- Returns raw audio (WAV format).
- Listens for a fixed duration (default 5s).

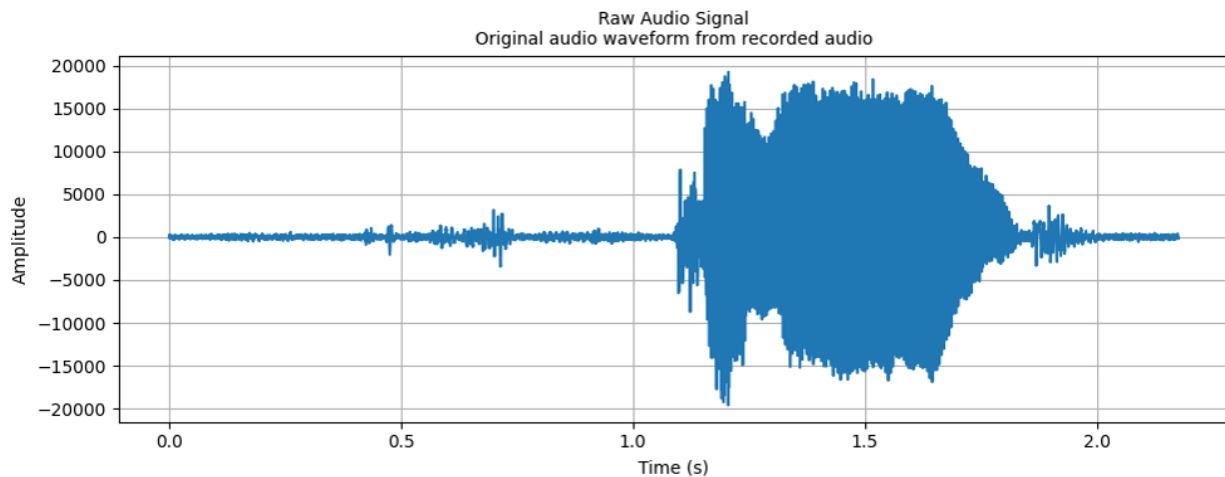


Fig 20: Recorded audio signal

7.2: Feature extraction

Converts raw audio into numeric features (MFCC). MFCCs (Mel-Frequency Cepstral Coefficients) are a set of features extracted from an audio signal, especially speech, that compactly represent how the human ear perceives sound. The human ear doesn't hear frequencies linearly (we're more sensitive to 500–4000 Hz). The Mel scale warps frequencies so they match human perception. "Cepstrum" is Fourier transform of the log spectrum. It separates the source (vocal cords) and the filter (vocal tract) in speech. After all transformations, we keep only a few numbers (typically 12–13 per frame). These numbers [array] summarize the spectral shape (the timbre) of the speech. The library used for MFCC operation is librosa.

```
def extract_voice_features(audio_data, sample_rate):
    """Extract MFCC features for speaker identification"""
    try:
        audio_np = np.frombuffer(audio_data, dtype=np.int16).astype(np.float32) / 32768.0
        if len(audio_np) < sample_rate // 10: # Ensure audio is long enough
            raise ValueError("Audio too short for reliable feature extraction")
        mfccs = librosa.feature.mfcc(y=audio_np, sr=sample_rate, n_mfcc=13)
        return np.mean(mfccs, axis=1) # Average MFCCs over time for a compact voiceprint
    except Exception as e:
        print(f"⚠ Voice Feature Extraction Error: {e}")
        return None
```

Fig 21: MFCC feature extraction code

How MFCC is computed

1. Pre-emphasis: Boost high frequencies to balance the spectrum.
2. Framing & Windowing: Split signal into short frames
3. FFT (Fourier Transform): Get frequency spectrum of each frame.
4. Apply Mel filter bank: Pass spectrum through a set of triangular filters spaced on the Mel scale.
5. Logarithm: Take the log of filter energies (mimics human loudness perception).
6. DCT (Discrete Cosine Transform): Decorrelates the log Mel spectrum which gives MFCCs.

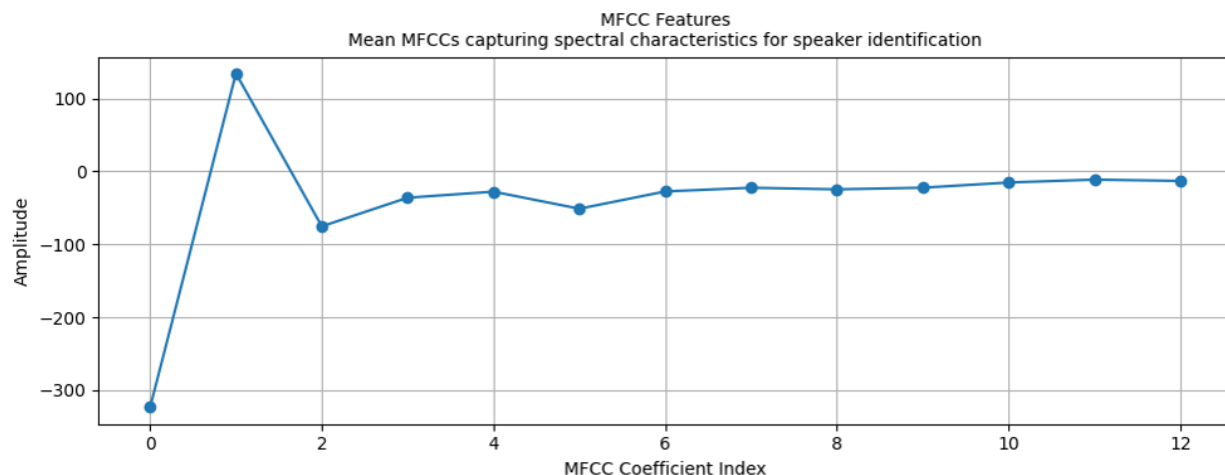


Fig 22: Extracted MFCC feature from recorded audio

7.3: Audio Matching

Compare new audio to saved ones using **Cosine similarity** between feature vectors. A threshold is set to 0.85. If the similarity is more than 0.85 it is a good match. Otherwise it didn't match. It finds the stored audio sample that is closest to the new one.

$$\text{cosine similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} \quad (7.3.1)$$

```
def match_audio(audio_data, sample_rate):
    try:
        input_features = extract_voice_features(audio_data, sample_rate)
        if input_features is None:
            return None
        if input_features.shape != (13,):
            raise ValueError(f"Invalid input feature shape: {input_features.shape}")
        audio_memory = load_audio_memory()
        if not audio_memory:
            print("⚠ No voiceprints available for matching")
            return None
        best_match = None
        best_similarity = -np.inf
        for desc, stored_features in audio_memory.items():
            stored_features = np.array(stored_features)
            if stored_features.shape != (13,):
                print(f"⚠ Skipping {desc}: Invalid stored feature shape {stored_features.shape}")
                continue
            similarity = 1 - cosine(input_features, stored_features)
            if similarity > best_similarity:
                best_similarity = similarity
                best_match = desc
                timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
                save_mfcc_plot(np.vstack([input_features, stored_features]).T,
                              f"MFCC Comparison (Input vs {desc})",
                              os.path.join(AUDIO_PLOT_DIR, f"mfcc_match_{timestamp}.png"),
                              description="Comparing input audio MFCCs to stored voiceprint (closer alignment = better mat
        return best_match if best_similarity > 0.85 else None
    except Exception as e:
        print(f"⚠ Voice Matching Error: {e}")
        return None
```

Fig 23: Code for audio matching

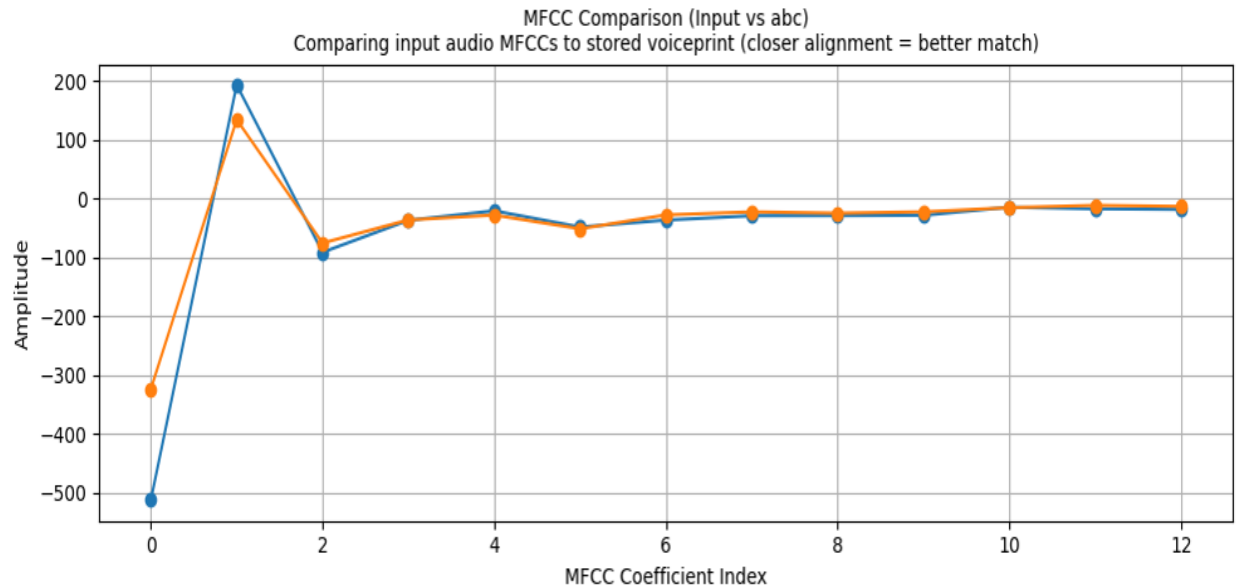


Fig 24: MFCC comparison of two recorded audios

7.4: Audio memory (save & load)

- Stores audio data + features in audio_memory.json.
- Allows recognition later
- Builds a database of sounds

```
def save_audio_memory(audio_data, sample_rate, description):
    """Save voice features to memory"""
    try:
        features = extract_voice_features(audio_data, sample_rate)
        if features is None:
            return "Failed to extract voice features"
        if features.shape != (13,):
            raise ValueError(f"Invalid feature shape: {features.shape}")
        audio_memory = load_audio_memory()
        audio_memory[description] = features.tolist()
        with open(AUDIO_MEMORY_FILE, "w") as f:
            json.dump(audio_memory, f, indent=2)
        return f"Saved voice with description: {description}"
    except Exception as e:
        return f"Failed to save voice to memory: {e}"

def load_audio_memory():
    """Load audio memory from file, validating feature shapes"""
    try:
        if os.path.exists(AUDIO_MEMORY_FILE):
            with open(AUDIO_MEMORY_FILE, "r") as f:
                audio_memory = json.load(f)
            # Validate feature shapes
            valid_memory = {}
            for desc, features in audio_memory.items():
                features = np.array(features)
                if features.shape == (13,):
                    valid_memory[desc] = features.tolist()
                else:
                    print(f"⚠ Invalid feature shape for {desc}: {features.shape}, discarding")
            return valid_memory
        return {}
    except Exception as e:
        print(f"⚠ Audio Memory Load Error: {e}")
        return {}
```

Fig 25: Audio memory (save & load) code

8. Additional features:

8.1: Open Apps / Websites:

- Respond to voice/text like: "Open YouTube" , "Launch Notepad"
- Uses `os.startfile` for apps.
- Uses `webbrowser.open` for URLs.
- Uses **fuzzy matching** so "ytube" still matches "youtube".

```
def open_command(command):
    global app_cache, app_index
    try:
        if "open" not in command: return None
        words = command.split("open",1)[1].strip().lower()
        all_choices = list(app_index.keys()) + list(dynamic_knowledge.keys()) + \
            ["youtube", "facebook", "google", "github", "twitter", "instagram", "reddit"]
        words = predict_word(words, all_choices)
        if words in app_cache:
            os.startfile(app_cache[words])
            return f"Opening {words}"
        if "." in words or "www" in words or words in all_choices:
            url = make_website_url(words)
            webbrowser.open(url)
            return f"Opening {url}"
        exe = shutil.which(words)
        if exe:
            subprocess.Popen([exe], shell=True)
            app_cache[words] = exe
            return f"Opening {words}"
        for key, path in app_index.items():
            if words in key:
                os.startfile(path)
                app_cache[words] = path
                return f"Opening {key}"
        return None
    except Exception as e:
        print(f"▲ Open Command Error: {e}")
        return None
```

Fig 26: Code for app / web page opening

8.2: Math & Physics

- Handle math problems & physics formulae directly.
- **Sympy** → algebra, calculus.
- **Custom parser** → physics word problems.

For example: "solve $x^2 - 4 = 0$ " → Returns $[-2, 2]$.

"find force when mass=10kg, acceleration=2m/s²" → Returns 20 N.

```

def solve_advanced_math(expression):
    try:
        expr = sympify(expression)
        if expr.is_number:
            return f"The solution is {expr}"
        result = solve(expr)
        return f"The solution is {result}"
    except:
        try:
            expr = sympify(expression)
            return f"Derivative: {diff(expr)}, Integral: {integrate(expr)}"
        except:
            return "Sorry, I could not solve that math problem."

def physics_calculator(question):
    try:
        if "force" in question and "mass" in question and "acceleration" in question:
            m = re.search(r"mass (\d+\.\d+)", question)
            a = re.search(r"acceleration (\d+\.\d+)", question)
            if m and a:
                return f"The force is {float(m.group(1))*float(a.group(1))} N"
        if "pressure" in question and "force" in question and "area" in question:
            F = re.search(r"force (\d+\.\d+)", question)
            A = re.search(r"area (\d+\.\d+)", question)
            if F and A:
                return f"The pressure is {float(F.group(1))/float(A.group(1))} Pa"
        if "speed" in question and "distance" in question and "time" in question:
            d = re.search(r"distance (\d+\.\d+)", question)
            t = re.search(r"time (\d+\.\d+)", question)
            if d and t:
                return f"The speed is {float(d.group(1))/float(t.group(1))} m/s"
        return None
    except Exception as e:
        print(f"⚠ Physics Calculation Error: {e}")
        return None

```

Fig 27: Code for solving math & physics problems

8.3: Google & Wikipedia Search

- Query the web when local knowledge is not enough.
- **Requests + BeautifulSoup** → scrape Google search results.
- **wikipedia API** → fetch summaries.

Example: "capital of Bangladesh" → Google result "Dhaka".

"Wikipedia: Quantum Mechanics" → Returns 2-sentence summary.

```

def google_search_summary(query, num_results=3):
    try:
        collected = []
        for url in search(query, num_results=num_results):
            try:
                r = requests.get(url, timeout=5, headers={"User-Agent": "Mozilla/5.0"})
                soup = BeautifulSoup(r.text, "html.parser")
                p = soup.find_all("p")
                for para in p[:2]:
                    text = para.get_text().strip()
                    if len(text) > 50:
                        collected.append(text)
                        break
            except:
                continue
        return " ".join(collected)[:500]+"..." if collected else None
    except Exception as e:
        print(f"⚠ Google Search Error: {e}")
        return None

def wiki_answer(query):
    try:
        return wikipedia.summary(query, sentences=2, auto_suggest=True, redirect=True)
    except Exception as e:
        print(f"⚠ Wikipedia Search Error: {e}")
        return None

```

Fig 28: Code for Google/Wikipedia search

8.4: Learning

Command format Learn that <question> is < answer>

Example: Command: Learn that **my name** is **Fahad**

Response: I have learned that **my name** is **Fahad**

```
if "learn that" in q_lower:
    try:
        fact = q_lower.replace("learn that","").strip()
        q,a = fact.split("is")
        dynamic_knowledge[q.strip()] = a.strip()
        save_memory()
        return f"I have learned that {q.strip()} is {a.strip()}"
    except:
        return "Sorry, use: 'Learn that [question] is [answer]'"
```

Fig 29: Code for learning

8.5: Knowledge base:

A knowledge base is set for basic introductions & Q&A

```
knowledge_base = {
    "your name": "I am your advanced voice assistant.",
    "who are you": "I am an AI assistant that can solve math, answer science questions, open apps/websites,",
    "hello": "Hello! How can I help you today?",
    "hi": "Hi there! What can I do for you?",
    "how are you": "I am functioning perfectly, thank you!",
    "time": f"The current time is {datetime.now().strftime('%I:%M %p')}",
    "date": f"Today is {datetime.now().strftime('%A, %d %B %Y')}",
    "bye": "Goodbye! Have a nice day!"
}
```

Fig 30: knowledge base

8.6: website builder:

```
def make_website_url(name):
    try:
        name = name.lower().replace(" ", "")
        if not name.startswith("http"):
            return f"https://www.{name}.com"
        return name
    except Exception as e:
        print(f"⚠ Website URL Error: {e}")
        return name
```

Fig 31: Code for website building

9. Discussion

The development of the advanced ai voice assistant integrated multiple domains of artificial intelligence, including speech recognition, natural language processing (NLP), digital signal processing (DSP), image analysis, and voice biometrics. By combining these domains into a multi-modal framework, the system moved beyond simple voice assistants that only answer spoken queries. Instead, it could understand commands, process multimedia inputs (voice and image), and recognize users by their unique voiceprints, creating a more personalized and capable assistant.

A major strength lies in the layered preprocessing pipeline for speech. Human speech is often corrupted by background noise, echoes, or microphone artifacts, which makes recognition unreliable. The pipeline first applied bandpass filtering to isolate frequencies relevant to human speech (300–3400 Hz). Then, FFT-based noise suppression removed low-energy components unlikely to belong to the speech signal. Finally, convolutional smoothing reduced sharp fluctuations. Together, these steps ensured the signal entering the recognition model was cleaner and more stable, which is especially valuable since the Google Speech Recognition API is known to degrade in noisy conditions. In addition to recognition, the assistant also extracted MFCC (Mel-Frequency Cepstral Coefficients), a widely used feature in speech and speaker recognition. By averaging MFCCs and comparing them using cosine similarity, the system could match a voice input with stored voiceprints. This demonstrated the assistant’s ability to not only understand what was being said but also identify who was speaking.

On the image processing side, the system employed two lightweight yet effective techniques:

- Histogram-based image matching: By computing grayscale histograms of stored and input images, the assistant compared them using correlation. This allowed recognition of previously saved images without requiring computationally heavy deep learning.
- Richardson–Lucy deconvolution: A classical iterative method for deblurring. It estimated the point-spread function (PSF) of blur and recovered sharper images.

Although these techniques are less advanced than CNN-based embeddings or GAN-based deblurring, they are computationally efficient, making them practical for real-time use without requiring GPU acceleration.

The knowledge system was designed as a hybrid, mirroring commercial assistants such as Siri and Alexa. It operated in three tiers:

1. Static knowledge base — For simple, frequently used responses (greetings, date, time), ensuring instant replies.
2. Dynamic memory module — Allowed the system to *learn* new facts during interaction and store them in JSON format, which gave it a sense of continuity and adaptability.
3. External knowledge retrieval — Wikipedia summaries and Google Search results expanded its coverage beyond predefined knowledge, enabling responses to a wide range of open-domain queries.

This architecture balances efficiency (quick retrieval for common tasks) with adaptability and scalability (external search for unknown queries).

Finally, the modular structure of the assistant promoted extensibility. Individual components (speech recognition, image capture, audio/image matching, math/physics solvers, app launching, web search) were designed as separate modules. This modularity meant that improvements or replacements could be made independently—for instance, swapping histogram matching with CNN-based embeddings, or upgrading the speech recognition model. The application indexer also enabled direct interaction with the operating system by locating installed programs, while web redirection handled online tasks. Together, these features ensured that the assistant could act as both a personal productivity tool and an information retrieval system, bridging local and online resources.

Limitations:

There are several limitations emerged during testing:

1. Real-time performance constraints

- Signal preprocessing (FFT, deblurring, MFCC extraction) introduces latency, particularly on resource-constrained hardware.
- Continuous voice matching and image feature comparisons are computationally expensive.

2. Dependence on external APIs

- Speech recognition relies heavily on Google's API. Offline recognition is limited, and network issues disrupt usability.
- Wikipedia and Google Search queries are susceptible to internet connectivity and rate-limiting.

3. Accuracy trade-offs in image processing

- Histogram-based similarity is relatively weak compared to deep CNN-based embeddings, making matching unreliable for complex scenes.
- Deblurring with Richardson–Lucy works for mild blur but fails with severe motion blur or noise.

4. Limited natural language understanding

- Command recognition relies on keyword matching rather than semantic parsing.
- Complex, multi-intent queries may fail or yield incomplete responses.

5. Security and privacy concerns

- Voiceprints and images are stored locally without encryption, creating potential vulnerabilities.
- No authentication layer exists to prevent unauthorized users from triggering actions.

6. Robustness to environmental variability

- Speech performance degrades in high-noise environments despite filtering.
- Voice matching thresholds may lead to false positives or false negatives, especially if the user's voice changes (illness, emotional tone).

10. Conclusion

The advanced voice assistant successfully demonstrated the integration of speech recognition, digital signal processing, NLP, image analysis, and voice biometrics into a single, modular system. Through layered preprocessing and feature extraction, it achieved robust speech recognition and user identification, while image matching and deblurring extended its functionality beyond conventional assistants. A hybrid knowledge system, combining static memory, dynamic learning, and external information retrieval, ensured both efficiency and adaptability. Although challenges remain in real-time performance, accuracy, and security, the project proved that combining multiple AI techniques within a modular framework can produce an assistant that is interactive, multimodal, and adaptive, bridging local productivity tasks with global information access.

11.References:

- [1] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A. Mohamed, N. Jaitly, et al., “Deep neural networks for acoustic modeling in speech recognition,” *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, Nov. 2012.
- [2] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, Aug. 1980.
- [3] C. Bishop, *Pattern Recognition and Machine Learning*. New York, NY: Springer, 2006.
- [4] A. P. Dempster, N. M. Laird, and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–38, 1977.
- [5] W. H. Richardson, “Bayesian-based iterative method of image restoration,” *Journal of the Optical Society of America*, vol. 62, no. 1, pp. 55–59, Jan. 1972.
- [6] L. B. Lucy, “An iterative technique for the rectification of observed distributions,” *The Astronomical Journal*, vol. 79, pp. 745–754, Jun. 1974.
- [7] J. Brownlee, *Deep Learning for Natural Language Processing*. San Francisco, CA: Machine Learning Mastery, 2017.
- [8] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Draft, Stanford University, 2023.
- [9] A. Graves, A. Mohamed, and G. Hinton, “Speech recognition with deep recurrent neural networks,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, Vancouver, Canada, May 2013, pp. 6645–6649.
- [10] R. Neff and A. Zakhor, “Very low bit-rate video coding based on matching pursuits,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 7, no. 1, pp. 158–171, Feb. 1997.

Full code:

```
import speech_recognition as sr

import pyttsx3

import numpy as np

from scipy.signal import butter, lfilter, convolve

from scipy.spatial.distance import cosine

from sympy import sympify, solve, diff, integrate

import re

import os

import webbrowser

import shutil

import subprocess

import json

import difflib

from datetime import datetime

import requests

from bs4 import BeautifulSoup

from googlesearch import search

import wikipedia

from PIL import Image

import io

import cv2

import time

import librosa

from skimage.restoration import richardson_lucy # Requires scikit-image

from scipy.ndimage import gaussian_filter
```

```

# ----- INITIALIZE -----

recognizer = sr.Recognizer()

app_cache = {}

app_index = {}

MEMORY_FILE = "memory.json"

IMAGE_MEMORY_FILE = "image_memory.json"

AUDIO_MEMORY_FILE = "audio_memory.json"

RECORDED_AUDIO_DIR = "recorded_audio"

CAPTURED_IMAGE_DIR = "captured_images"


# Create directories

if not os.path.exists(RECORDED_AUDIO_DIR):
    os.makedirs(RECORDED_AUDIO_DIR)

if not os.path.exists(CAPTURED_IMAGE_DIR):
    os.makedirs(CAPTURED_IMAGE_DIR)


# ----- TTS SETUP -----

def speak(text):
    try:
        engine = pyttsx3.init()
        engine.setProperty('rate', 150)
        engine.setProperty('volume', 1.0)
        engine.say(text)
        engine.runAndWait()
    except Exception as e:

```

```
print(f" ⚠ TTS Error: {e}")
```

```
# ----- DSP FUNCTIONS -----
```

```
def butter_bandpass(lowcut, highcut, fs, order=5):
```

```
    nyq = 0.5 * fs
```

```
    low = lowcut / nyq
```

```
    high = highcut / nyq
```

```
    b, a = butter(order, [low, high], btype='band')
```

```
    return b, a
```

```
def bandpass_filter(data, lowcut=300, highcut=3400, fs=16000, order=5):
```

```
    b, a = butter_bandpass(lowcut, highcut, fs, order=order)
```

```
    return lfilter(b, a, data)
```

```
def preprocess_audio(audio_data, sample_rate):
```

```
    try:
```

```
        audio_np = np.frombuffer(audio_data, dtype=np.int16)
```

```
        filtered = bandpass_filter(audio_np, 300, 3400, fs=sample_rate, order=6)
```

```
        fft_signal = np.fft.fft(filtered)
```

```
        magnitude = np.abs(fft_signal)
```

```
        threshold = np.max(magnitude) * 0.01
```

```
        fft_signal[magnitude < threshold] = 0
```

```
        filtered_fft = np.fft.ifft(fft_signal).real
```

```
        kernel = np.ones(5)/5
```

```

smoothed = convolve(filtered_fft, kernel, mode="same")

voice_indices = np.where(np.abs(smoothed) > 500)[0]
if len(voice_indices) > 0:
    smoothed = smoothed[voice_indices[0]:voice_indices[-1]+1]

return smoothed.astype(np.int16).tobytes(), smoothed
except Exception as e:
    print(f" ⚠ Audio Processing Error: {e}")
    return None, None

# ----- IMAGE PROCESSING -----
def extract_image_features(image_data):
    """Extract histogram features for image matching with resized image"""
    try:
        img = cv2.imdecode(np.frombuffer(image_data, np.uint8), cv2.IMREAD_GRAYSCALE)
        # Resize image to reduce processing time (max width/height 300 pixels)
        max_size = 300
        height, width = img.shape
        scale = max_size / max(height, width)
        if scale < 1:
            img = cv2.resize(img, (int(width * scale), int(height * scale)),
interpolation=cv2.INTER_AREA)
        hist = cv2.calcHist([img], [0], None, [256], [0, 256])
        cv2.normalize(hist, hist)
        return hist.flatten().tolist()

```

```
except Exception as e:
```

```
    print(f" ⚠ Image Feature Extraction Error: {e}")
```

```
    return None
```

```
def match_image(image_data):
```

```
    """Match input image against stored image features"""
```

```
    try:
```

```
        input_features = extract_image_features(image_data)
```

```
        if input_features is None:
```

```
            return None
```

```
        input_hist = np.array(input_features, dtype=np.float32)
```

```
        image_memory = load_image_memory()
```

```
        if not image_memory:
```

```
            print(" ⚠ No image features available for matching")
```

```
            return None
```

```
        best_match = None
```

```
        best_similarity = -np.inf
```

```
        for desc, data in image_memory.items():
```

```
            stored_hist = np.array(data.get("features"), dtype=np.float32)
```

```
            similarity = cv2.compareHist(input_hist, stored_hist, cv2.HISTCMP_CORREL)
```

```
            if similarity > best_similarity:
```

```
                best_similarity = similarity
```

```
                best_match = desc
```

```

        return best_match if best_similarity > 0.8 else None
except Exception as e:

    print(f" ⚠ Image Matching Error: {e}")

    return None


def capture_image(image_source="webcam"):

    """Capture image from webcam or file, save to file, and display"""

    try:

        if image_source == "webcam":

            cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)

            if not cap.isOpened():

                return None, "Failed to open webcam. Ensure it is connected and not in use."

            time.sleep(1)

            ret, frame = cap.read()

            cap.release()

            if not ret:

                return None, "Failed to capture image from webcam"

            # Save to file

            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

            filename = os.path.join(CAPTURED_IMAGE_DIR, f"captured_{timestamp}.jpg")

            cv2.imwrite(filename, frame)

            # Display on screen

            cv2.imshow("Captured Image", frame)

            cv2.waitKey(0) # Wait for any key press to close

            cv2.destroyAllWindows()

```

```

_, buffer = cv2.imencode('.jpg', frame)

    return buffer.tobytes(), f"Image captured from webcam, saved as {filename}, and
displayed on screen"

else:

    if os.path.exists(image_source):

        img = cv2.imread(image_source)

        if img is None:

            return None, f"Failed to load image from {image_source}"

        _, buffer = cv2.imencode('.jpg', img)

        return buffer.tobytes(), f"Image loaded from {image_source}"

    return None, f"File {image_source} not found"

except Exception as e:

    return None, f"Image Capture Error: {e}"

def deblur_image(description):

    """Apply Richardson-Lucy deconvolution to deblur an image from memory and display it"""

    try:

        image_memory = load_image_memory()

        if description not in image_memory:

            return None, f"No image found for description: {description}"

        image_data = bytes.fromhex(image_memory[description]["data"])

        img = cv2.imdecode(np.frombuffer(image_data, np.uint8), cv2.IMREAD_COLOR) # Load
in color

        if img is None:

            return None, f"Failed to decode image for description: {description}"

        # Convert to float32 and normalize to [0, 1] for each channel

```

```

img = img.astype(np.float32) / 255.0

# Create Gaussian blur kernel
kernel_size = 5

sigma = kernel_size / 4.0 # Smaller sigma for sharper PSF
psf = np.zeros((kernel_size, kernel_size))
center = kernel_size // 2
for i in range(kernel_size):
    for j in range(kernel_size):
        psf[i, j] = np.exp(-((i - center) ** 2 + (j - center) ** 2) / (2 * sigma ** 2))
psf /= psf.sum() # Normalize PSF

# Apply Richardson-Lucy deconvolution to each color channel
deblurred = np.zeros_like(img)
for channel in range(img.shape[2]):
    deblurred[:, :, channel] = richardson_lucy(
        img[:, :, channel], psf, num_iter=15, clip=True
    )

# Clip and scale back to uint8
deblurred = np.clip(deblurred * 255, 0, 255).astype(np.uint8)

# Apply sharpening filter
sharpening_kernel = np.array([
    [0, -1, 0],
    [-1, 5, -1],

```

```

        [0, -1, 0]

    ])

    deblurred = cv2.filter2D(deblurred, -1, sharpening_kernel)

    # Display original and deblurred images for comparison

    original_img = cv2.imdecode(np.frombuffer(image_data, np.uint8),
cv2.IMREAD_COLOR)

    cv2.imshow("Original Image", original_img)

    cv2.imshow("Deblurred Image", deblurred)

    cv2.waitKey(0) # Wait for any key press to close

    cv2.destroyAllWindows()

    _, buffer = cv2.imencode('.jpg', deblurred)

    return buffer.tobytes(), f"Deblurred image for {description} and displayed on screen"
except Exception as e:

    print(f" ⚠ Image Clear Error: {e}")

    return None, f"Image Clear Error: {e}"

def save_image_memory(image_data, description):

    """Save processed image and its features to memory"""

    try:

        image_memory = load_image_memory()

        features = extract_image_features(image_data)

        if features is None:

            return f"Failed to save image: Could not extract features"

        image_memory[description] = {

```

```

        "data": image_data.hex(),
        "features": features
    }
    with open(IMAGE_MEMORY_FILE, "w") as f:
        json.dump(image_memory, f, indent=2)
    return f"Saved image with description: {description}"
except Exception as e:
    return f"Failed to save image to memory: {e}"

def load_image_memory():
    """Load image memory from file"""
    try:
        if os.path.exists(IMAGE_MEMORY_FILE):
            with open(IMAGE_MEMORY_FILE, "r") as f:
                image_memory = json.load(f)
            # Validate entries
            valid_memory = {}
            for desc, data in image_memory.items():
                if isinstance(data, dict) and "data" in data and "features" in data:
                    valid_memory[desc] = data
                else:
                    print(f"⚠ Invalid image memory entry for {desc}, discarding")
            return valid_memory
        return {}
    except Exception as e:
        print(f"⚠ Image Memory Load Error: {e}")

```

```

    return {}

# ----- VOICE RECORDING -----

def record_audio(sample_rate=16000):
    """Record audio until silence is detected"""
    try:
        with sr.Microphone(sample_rate=sample_rate) as mic_source:
            recognizer.adjust_for_ambient_noise(mic_source)
            print("Recording... Speak now (stops after silence).")
            audio = recognizer.listen(mic_source, timeout=5, phrase_time_limit=None)
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = os.path.join(RECORDED_AUDIO_DIR, f"recording_{timestamp}.wav")
            with open(filename, "wb") as f:
                f.write(audio.get_wav_data())
            return audio.get_raw_data(), sample_rate, f"Audio recorded and saved as {filename}"
    except Exception as e:
        print(f" ⚠ Audio Recording Error: {e}")
        return None, None, f"Failed to record audio: {e}"

# ----- VOICE MATCHING -----

def extract_voice_features(audio_data, sample_rate):
    """Extract MFCC features for speaker identification"""
    try:
        audio_np = np.frombuffer(audio_data, dtype=np.int16).astype(np.float32) / 32768.0
        if len(audio_np) < sample_rate // 10: # Ensure audio is long enough
            raise ValueError("Audio too short for reliable feature extraction")
    
```

```

mfccs = librosa.feature.mfcc(y=audio_np, sr=sample_rate, n_mfcc=13)

return np.mean(mfccs, axis=1) # Average MFCCs over time for a compact voiceprint
except Exception as e:

    print(f" ⚠ Voice Feature Extraction Error: {e}")

    return None

```

```

def save_audio_memory(audio_data, sample_rate, description):

```

```

    """Save voice features to memory"""

    try:

        features = extract_voice_features(audio_data, sample_rate)

        if features is None:

            return "Failed to extract voice features"

        if features.shape != (13,):

            raise ValueError(f"Invalid feature shape: {features.shape}")

        audio_memory = load_audio_memory()

        audio_memory[description] = features.tolist()

        with open(AUDIO_MEMORY_FILE, "w") as f:

            json.dump(audio_memory, f, indent=2)

        return f"Saved voice with description: {description}"

    except Exception as e:

        return f"Failed to save voice to memory: {e}"

```

```

def load_audio_memory():

```

```

    """Load audio memory from file, validating feature shapes"""

    try:

        if os.path.exists(AUDIO_MEMORY_FILE):

```

```

with open(AUDIO_MEMORY_FILE, "r") as f:
    audio_memory = json.load(f)
    # Validate feature shapes
    valid_memory = {}
    for desc, features in audio_memory.items():
        features = np.array(features)
        if features.shape == (13,):
            valid_memory[desc] = features.tolist()
        else:
            print(f" ⚠ Invalid feature shape for {desc}: {features.shape}, discarding")
    return valid_memory
return {}
except Exception as e:
    print(f" ⚠ Audio Memory Load Error: {e}")
    return {}

def match_audio(audio_data, sample_rate):
    """Match input voice against stored voiceprints"""
    try:
        input_features = extract_voice_features(audio_data, sample_rate)
        if input_features is None:
            return None
        if input_features.shape != (13,):
            raise ValueError(f"Invalid input feature shape: {input_features.shape}")
        audio_memory = load_audio_memory()
        if not audio_memory:

```

```

print(" ⚠ No voiceprints available for matching")

return None

best_match = None

best_similarity = -np.inf

for desc, stored_features in audio_memory.items():

    stored_features = np.array(stored_features)

    if stored_features.shape != (13,):

        print(f" ⚠ Skipping {desc}: Invalid stored feature shape {stored_features.shape}")

        continue

    similarity = 1 - cosine(input_features, stored_features) # Cosine similarity

    if similarity > best_similarity:

        best_similarity = similarity

        best_match = desc

return best_match if best_similarity > 0.85 else None # Threshold for voice match
except Exception as e:

    print(f" ⚠ Voice Matching Error: {e}")

    return None

# ----- MATH -----

def solve_advanced_math(expression):

    try:

        expr = sympify(expression)

        if expr.is_number:

```

```

        return f"The solution is {expr}"

    result = solve(expr)

    return f"The solution is {result}"

except:

    try:

        expr = sympify(expression)

        return f"Derivative: {diff(expr)}, Integral: {integrate(expr)}"

    except:

        return "Sorry, I could not solve that math problem."

# ----- PHYSICS -----

def physics_calculator(question):

    try:

        if "force" in question and "mass" in question and "acceleration" in question:

            m = re.search(r"mass (\d+\.\d*)", question)

            a = re.search(r"acceleration (\d+\.\d*)", question)

            if m and a:

                return f"The force is {float(m.group(1))*float(a.group(1))} N"

        if "pressure" in question and "force" in question and "area" in question:

            F = re.search(r"force (\d+\.\d*)", question)

            A = re.search(r"area (\d+\.\d*)", question)

            if F and A:

                return f"The pressure is {float(F.group(1))/float(A.group(1))} Pa"

        if "speed" in question and "distance" in question and "time" in question:

```

```

d = re.search(r"distance (\d+\.\?\d*)", question)
t = re.search(r"time (\d+\.\?\d*)", question)

if d and t:
    return f"The speed is {float(d.group(1))/float(t.group(1))} m/s"

return None

except Exception as e:
    print(f" ⚠️ Physics Calculation Error: {e}")
    return None

```

----- SEARCH -----

```

def google_search_summary(query, num_results=3):
    try:
        collected = []
        for url in search(query, num_results=num_results):
            try:
                r = requests.get(url, timeout=5, headers={"User-Agent": "Mozilla/5.0"})
                soup = BeautifulSoup(r.text, "html.parser")
                p = soup.find_all("p")
                for para in p[:2]:
                    text = para.get_text().strip()
                    if len(text) > 50:
                        collected.append(text)
                        break
            except:
                continue
        return " ".join(collected)[:500]+"..." if collected else None

```

```

except Exception as e:

    print(f" ⚠️ Google Search Error: {e}")

    return None


def wiki_answer(query):

    try:

        return wikipedia.summary(query, sentences=2, auto_suggest=True, redirect=True)

    except Exception as e:

        print(f" ⚠️ Wikipedia Search Error: {e}")

        return None


# ----- MEMORY -----

def load_memory():

    try:

        if os.path.exists(MEMORY_FILE):

            with open(MEMORY_FILE, "r") as f:

                return json.load(f)

        return {}

    except Exception as e:

        print(f" ⚠️ Memory Load Error: {e}")

        return {}


def save_memory():

    try:

        with open(MEMORY_FILE, "w") as f:

            json.dump(dynamic_knowledge, f, indent=2)

```

```

except Exception as e:

    print(f" ⚠ Memory Save Error: {e}")


dynamic_knowledge = load_memory()


# ----- APP INDEX -----

def build_app_index():

    global app_index

    try:

        common_paths = [

            os.path.join(os.environ["APPDATA"], r"Microsoft\Windows\Start Menu\Programs"),

            os.path.join(os.environ["ProgramData"], r"Microsoft\Windows\Start Menu\Programs"),

        ]

        for base in common_paths:

            if not os.path.exists(base): continue

            for root, dirs, files in os.walk(base):

                for f in files:

                    if f.lower().endswith(('.lnk', '.exe')):

                        key = os.path.splitext(f)[0].lower()

                        app_index[key] = os.path.join(root, f)

    except Exception as e:

        print(f" ⚠ App Index Build Error: {e}")


# ----- WORD PREDICTION -----

def predict_word(word, choices):

    try:

```

```

    matches = difflib.get_close_matches(word, choices, n=1, cutoff=0.6)

    return matches[0] if matches else word

except Exception as e:

    print(f" ⚠ Word Prediction Error: {e}")

    return word


# ----- WEBSITE BUILDER -----

def make_website_url(name):

    try:

        name = name.lower().replace(" ", "")

        if not name.startswith("http"):

            return f"https://www.{name}.com"

        return name

    except Exception as e:

        print(f" ⚠ Website URL Error: {e}")

        return name


# ----- OPEN COMMAND -----

def open_command(command):

    global app_cache, app_index

    try:

        if "open" not in command: return None

        words = command.split("open",1)[1].strip().lower()

        all_choices = list(app_index.keys()) + list(dynamic_knowledge.keys()) + \

            ["youtube","facebook","google","github","twitter","instagram","reddit"]

```

```

words = predict_word(words, all_choices)

if words in app_cache:
    os.startfile(app_cache[words])
    return f"Opening {words}"

if "." in words or "www" in words or words in all_choices:
    url = make_website_url(words)
    webbrowser.open(url)
    return f"Opening {url}"

exe = shutil.which(words)
if exe:
    subprocess.Popen([exe], shell=True)
    app_cache[words] = exe
    return f"Opening {words}"

for key, path in app_index.items():
    if words in key:
        os.startfile(path)
        app_cache[words] = path
        return f"Opening {key}"

return None

except Exception as e:
    print(f"⚠️ Open Command Error: {e}")

```

```

return None

# ----- KNOWLEDGE BASE -----

knowledge_base = {
    "your name": "I am your advanced voice assistant.",
    "who are you": "I am an AI assistant that can solve math, answer science questions, open
apps/websites, capture images, and match voices and images.",
    "hello": "Hello! How can I help you today?",
    "hi": "Hi there! What can I do for you?",
    "how are you": "I am functioning perfectly, thank you!",
    "time": f"The current time is {datetime.now().strftime('%l:%M %p')}",
    "date": f"Today is {datetime.now().strftime('%A, %d %B %Y')}",
    "bye": "Goodbye! Have a nice day!"
}

# ----- GET ANSWER -----

def get_answer(question, audio_data=None, sample_rate=None, image_data=None):
    try:
        q_lower = question.lower()

        # Memory
        if q_lower in dynamic_knowledge:
            return dynamic_knowledge[q_lower]

        # Image capture
        if "capture image" in q_lower:

```

```

image_source = "webcam"

if "from file" in q_lower:

    image_source = q_lower.split("from file")[-1].strip()

image_data, message = capture_image(image_source)

if image_data:

    description = q_lower.replace("capture image", "").replace("from file", "").strip() or
"captured_image"

    save_image_memory(image_data, description)

    return message

return message


# Image matching

if q_lower.startswith("match image"):

    if image_data:

        match = match_image(image_data)

        if match and match in load_image_memory():

            return f"Image matched to: {match}"

        return "Didn't match"

    return "No image data available for matching"


# Image processing commands

if "clear image" in q_lower:

    description = q_lower.replace("clear image", "").strip() or "cleared_image"

    cleared, message = deblur_image(description)

    if cleared:

        save_image_memory(cleared, description)

```

```

        return message

    return message

# Audio recording
if q_lower.startswith("record audio"):
    audio_data, sample_rate, message = record_audio()

    if audio_data:
        description = q_lower.replace("record audio", "").strip() or
f"voice_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
        save_audio_memory(audio_data, sample_rate, description)

        return message

    return message

# Audio matching
if q_lower.startswith("match audio"):
    if audio_data and sample_rate:
        match = match_audio(audio_data, sample_rate)

        if match and match in load_audio_memory():
            return f"Audio matched to: {match}"

        return "Didn't match"

    return "No audio data available for matching"

# Open apps/web
open_result = open_command(q_lower)

if open_result: return open_result

```

```
# Math
```

```
if any(w in q_lower for w in ["calculate","solve","evaluate","derivative","integral"]):
```

```
    expr = q_lower
```

```
    for w in ["calculate","solve","evaluate","find derivative of","integral of"]:
```

```
        expr = expr.replace(w,"")
```

```
    result = solve_advanced_math(expr.strip())
```

```
    dynamic_knowledge[q_lower] = result; save_memory()
```

```
    return result
```

```
# Physics
```

```
physics_result = physics_calculator(q_lower)
```

```
if physics_result:
```

```
    dynamic_knowledge[q_lower] = physics_result; save_memory()
```

```
    return physics_result
```

```
# Knowledge base
```

```
for key in knowledge_base:
```

```
    if key in q_lower: return knowledge_base[key]
```

```
# Learn
```

```
if "learn that" in q_lower:
```

```
    try:
```

```
        fact = q_lower.replace("learn that","").strip()
```

```
        q,a = fact.split("is")
```

```
        dynamic_knowledge[q.strip()] = a.strip()
```

```
        save_memory()
```

```

        return f"I have learned that {q.strip()} is {a.strip()}"
    except:
        return "Sorry, use: 'Learn that [question] is [answer]'"

# Wiki
wiki_result = wiki_answer(question)
if wiki_result:
    dynamic_knowledge[q_lower] = wiki_result; save_memory()
    return wiki_result

# Google
google_result = google_search_summary(question)
if google_result:
    dynamic_knowledge[q_lower] = google_result; save_memory()
    return google_result

return "Sorry, I don't know that yet."
except Exception as e:
    print(f" 🚨 Get Answer Error: {e}")
    return f"Error processing command: {e}"

# ----- MAIN -----

print(" 🎤 Advanced Voice Assistant running... Say 'bye' to exit.")
build_app_index()

# Clear audio_memory.json to ensure clean state

```

```

if os.path.exists(AUDIO_MEMORY_FILE):
    os.remove(AUDIO_MEMORY_FILE)
    print(f"Cleared {AUDIO_MEMORY_FILE} to ensure clean state.")

# Clear image_memory.json to ensure clean state for new format
if os.path.exists(IMAGE_MEMORY_FILE):
    os.remove(IMAGE_MEMORY_FILE)
    print(f"Cleared {IMAGE_MEMORY_FILE} to ensure clean state for new format.")

while True:
    try:
        # Audio processing
        audio_data = None
        sample_rate = None
        with sr.Microphone() as mic_source:
            if not hasattr(mic_source, 'SAMPLE_RATE'):
                print("⚠ Microphone initialization failed. Retrying...")
                continue
            recognizer.adjust_for_ambient_noise(mic_source)
            print("\nListening...")
            audio = recognizer.listen(mic_source, timeout=5, phrase_time_limit=5)

            processed, audio_np = preprocess_audio(audio.get_raw_data(),
mic_source.SAMPLE_RATE)
            if processed is None:
                print("❌ Audio processing failed")

```

```

        continue

        audio_data = sr.AudioData(processed, mic_source.SAMPLE_RATE,
mic_source.SAMPLE_WIDTH)

        sample_rate = mic_source.SAMPLE_RATE

        text = recognizer.recognize_google(audio_data)

        print(f"✅ You said: {text}")

# Image processing (only for match image)
image_data = None
if "match image" in text.lower():
    image_source = "webcam"
    if "from file" in text.lower():
        image_source = text.lower().split("from file")[-1].strip()
    image_data, image_message = capture_image(image_source)
    if image_data is None:
        print(f"⚠️ Image Capture Failed: {image_message}")
        speak(image_message)
        continue

    answer = get_answer(text, audio_data.get_raw_data(), sample_rate, image_data)
    print(f"Assistant: {answer}")
    speak(answer)

if "bye" in text.lower():

```

```
break
```

```
except sr.UnknownValueError:
```

```
    print("❌ Could not understand audio")
```

```
except sr.RequestError as e:
```

```
    print(f"⚠️ Speech recognition service error: {e}")
```

```
    speak(f"Speech recognition service error: {e}")
```

```
except Exception as e:
```

```
    print(f"⚠️ Error: {e}")
```

```
    speak("Something went wrong, but I am still running.")
```